Laboratório de Concorrência · Aula 10

Sérgio Guilherme Reis Pedroso · RA 24139932-2 · Turma ESOFT6S-N-B
Gerado em 09/10/2026, 20:21:46 · tempo registrado no cronômetro: 29:59

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma pessoa só consegue continuar depois que a outra termina a parte dela. Num restaurante, o garçom espera o cozinheiro deixar o prato pronto para então levar na mesa. Já a sincronização de competição acontece quando duas pessoas querem usar a mesma coisa ao mesmo tempo e só uma pode usar. Se há um banheiro com uma cabine só, uma pessoa entra e a outra espera do lado de fora, porque as duas querem o mesmo lugar e não dá para usar junto.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero que o programa imprima 2000000

1ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1043698

2ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1351776

3ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1369674

Saída do programa

```
esperado: 2000
obtido:   2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores não bateram com o esperado e também não foram iguais nas três execuções. Em cada execução as duas se cruzam em momentos diferentes, então a quantidade de somas que se perdem também muda.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

As duas threads leem o mesmo valor do contador, por exemplo 10, antes de qualquer uma gravar. Cada uma soma 1 no próprio registrador e fica com 11. A thread A grava 11 na memória e, em seguida, a thread B grava 11 por cima. As duas fizeram o trabalho, mas o contador só andou um, porque a segunda gravação apagou a primeira.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000 o resultado costuma vir certo, 2000, porque o trecho é tão curto que muitas vezes as threads quase não se cruzam.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

BBBAAA → 7

ABABAB → 6

BABABA → 4

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis e só 4 resultados diferentes: 4, 6, 7 e 8.
o 8 aparece em uma ordem só, a AAABBB, quando A faz tudo antes de B começar

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA, A lê o TOTAL, que vale 3, e calcula 4 no registrador dela. B também lê 3, multiplica por 2, fica com 6 e grava 6 na memória. Aí A grava o 4 que estava no registrador dela e apaga o 6. A conta de B foi feita, mas se perdeu, porque A escreveu por último com o valor antigo.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Eu acho que a versão sem sincronização vai ser a mais rápida e a única incorreta

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1591584 em 4 ms
synchronized:      2000000 em 75 ms
AtomicInteger:     2000000 em 23 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

O synchronized e o AtomicInteger deram 2000000. A versão sem sincronização ficou abaixo disso. As versões corretas demoram mais porque cada soma precisa se entender com a outra thread: no synchronized a thread pega e solta uma trava a cada incremento, e no AtomicInteger o processador faz ler, somar e gravar como um passo só.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Um AtomicInteger em cada saldo deixa cada conta segura sozinha, mas a transferência mexe nas duas: tira de uma e põe na outra. Entre esses dois passos outra operação pode entrar, e por um instante o dinheiro já saiu de um lado e ainda não chegou no outro. Para a transferência valer como uma coisa só, as duas alterações precisam ficar na mesma seção crítica, por exemplo com synchronized pegando as duas contas sempre na mesma ordem.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

As mensagens "A terminou" e "B terminou" não vão aparecer

Saídas (antes e depois da correção)

```
Antes:
IMPASSE: 2 threads presas

Depois:
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

thread A ficou com o X e esperava o Y. A thread B ficou com o Y e esperava o X. Cada uma segurava justamente o recurso que a outra precisava, e nenhuma solta o que já pegou, então as duas esperam para sempre e o trecho que imprimiria "terminou" nunca roda.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se todo mundo pega primeiro o X e só depois o Y, quem chega em segundo espera o X ainda de mãos vazias. Ninguém segura o Y enquanto espera o X, e o ciclo não fecha. Num banco, uma transferência da conta 1 para a 2 e outra da 2 para a 1 travam se cada uma trava primeiro a conta de origem.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores eu espero cerca de 200 ms, porque cada um faz duas tarefas de 100 ms. Com 1 trabalhador cerca de 600 ms. Com 6 cerca de 100 ms.

Tempo com 1 trabalhador

200ms

Tempo com 3 trabalhadores

600ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 5 fez a tarefa 6
trabalhador 6 fez a tarefa 1
```

```
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 2 fez a tarefa 3
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador são seis tarefas uma atrás da outra, então o tempo fica perto de 600 ms. Com 3, o serviço se divide em duas rodadas e cai para cerca de 200 ms. Com 6, cada um faz uma só e o total fica perto de 100 ms. Não precisa de trava porque a tarefa vai numa mensagem do canal e só um trabalhador recebe cada uma. Ninguém altera o mesmo dado ao mesmo tempo: o trabalho é entregue, em vez de ficar numa variável compartilhada.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2

depois:
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem lugar nenhum para guardar a mensagem. O envio do 1 fica esperando alguém receber, mas o recebimento está na mesma rotina e a execução nunca chega lá. Com espaço para 1, o envio entra no canal e o programa recebe esse valor e imprime 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Eu acho que o programa nem chega a rodar.

Saídas (o erro e o resultado com move)

```
sem move:
Non UTF-8 characters found in the code


com move:
0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread leva uma cópia do 0, não a variável do main. Ela soma 1 nessa cópia, que vira 1 e desaparece quando a thread acaba. O contador do main continua 0, e é esse valor que o println mostra.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java o erro só aparece com o programa rodando, e numa execução de teste o resultado ainda pode vir certo. Em Rust o compilador segura o erro antes de executar, quando a thread tenta alterar a variável do main sem proteção. A vantagem é achar a corrida na compilação, sem depender do escalonador. O custo é escrever mais: para compartilhar de verdade entra Arc com Mutex, um clone para cada thread e uma trava a cada soma.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, B, C e D.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 301 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram A, D, C, B. A e D fazem parte do código que já está rodando, então imprimem na hora, A primeiro e D logo em seguida. B e C ficam numa fila e só entram quando esse trecho termina. C é uma promessa, e essa fila passa na frente da fila do setTimeout, então C sai antes de B mesmo com o tempo de B marcado em 0.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer E era de 10 ms, mas a função ocupado segurou a única thread por cerca de 300 ms num laço que não devolve o controle. Enquanto esse laço roda, o laço de eventos não consegue chamar mais nada, e o E só dispara quando os 300 ms acabam. Num servidor Node.js, um cálculo pesado desse tipo segura todos os usuários ao mesmo tempo, porque não há outra thread para atender o resto no meio do caminho.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

pode ocorrer uma condição de corrida. É possível evitar realizando a reserva e atualização de estoque com uma transação atômica no banco de dados.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

foi ver que, de 20 jeitos de intercalar as duas tarefas, só um produz o resultado que a gente chama de certo.